

EXAMPLE

ANSWER

programmable logic array (PLA)
A structured-logic element composed of a set of inputs and corresponding input complements and two stages of logic: the first generates product terms of the inputs and input complements, and the second generates sum terms of the product terms. Hence, PLAs implement logic functions as a sum of products.

minterms Also called **product terms**. A set of logic inputs joined by conjunction (AND operations); the product terms form the first logic stage of the *programmable logic array* (PLA).

Sum of Products

Show the sum-of-products representation for the following truth table for *D*.

Inputs		Outputs	
A	B	C	D
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

There are four product terms, since the function is true (1) for four different input combinations. These are:

$$\begin{aligned} &\bar{A} \cdot \bar{B} \cdot C \\ &\bar{A} \cdot B \cdot C \\ &A \cdot \bar{B} \cdot \bar{C} \\ &A \cdot B \cdot C \end{aligned}$$

Thus, we can write the function for *D* as the sum of these terms:

$$D = (\bar{A} \cdot \bar{B} \cdot C)(\bar{A} \cdot B \cdot C)(A \cdot \bar{B} \cdot \bar{C})(A \cdot B \cdot C)$$

Note that only those truth table entries for which the function is true generate terms in the equation.

We can use this relationship between a truth table and a two-level representation to generate a gate-level implementation of any set of logic functions. A set of logic functions corresponds to a truth table with multiple output columns, as we saw in the example on page A-5. Each output column represents a different logic function, which may be directly constructed from the truth table.

The sum-of-products representation corresponds to a common structured-logic implementation called a **programmable logic array (PLA)**. A PLA has a set of inputs and corresponding input complements (which can be implemented with a set of inverters), and two stages of logic. The first stage is an array of AND gates that form a set of **product terms** (sometimes called **minterms**); each product term can consist of any of the inputs or their complements. The second stage is an array of OR gates, each of which forms a logical sum of any number of the product terms. [Figure A.3.3](#) shows the basic form of a PLA.

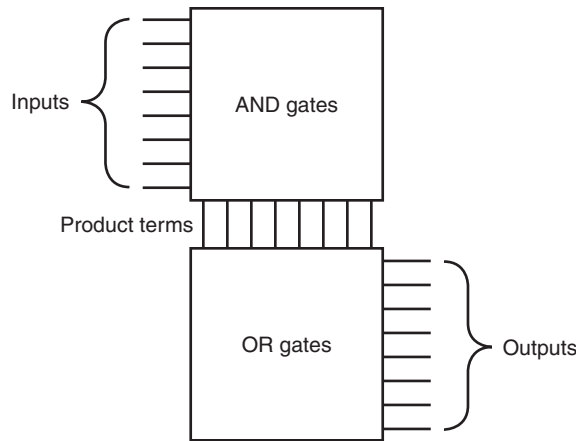


FIGURE A.3.3 The basic form of a PLA consists of an array of AND gates followed by an array of OR gates. Each entry in the AND gate array is a product term consisting of any number of inputs or inverted inputs. Each entry in the OR gate array is a sum term consisting of any number of these product terms.

A PLA can directly implement the truth table of a set of logic functions with multiple inputs and outputs. Since each entry where the output is true requires a product term, there will be a corresponding row in the PLA. Each output corresponds to a potential row of OR gates in the second stage. The number of OR gates corresponds to the number of truth table entries for which the output is true. The total size of a PLA, such as that shown in Figure A.3.3, is equal to the sum of the size of the AND gate array (called the *AND plane*) and the size of the OR gate array (called the *OR plane*). Looking at Figure A.3.3, we can see that the size of the AND gate array is equal to the number of inputs times the number of different product terms, and the size of the OR gate array is the number of outputs times the number of product terms.

A PLA has two characteristics that help make it an efficient way to implement a set of logic functions. First, only the truth table entries that produce a true value for at least one output have any logic gates associated with them. Second, each different product term will have only one entry in the PLA, even if the product term is used in multiple outputs. Let's look at an example.

PLAs

Consider the set of logic functions defined in the example on page A-5. Show a PLA implementation of this example for D , E , and F .

EXAMPLE

ANSWER

Here is the truth table we constructed earlier:

Inputs			Outputs		
A	B	C	D	E	F
0	0	0	0	0	0
0	0	1	1	0	0
0	1	0	1	0	0
0	1	1	1	1	0
1	0	0	1	0	0
1	0	1	1	1	0
1	1	0	1	1	0
1	1	1	1	0	1

Since there are seven unique product terms with at least one true value in the output section, there will be seven columns in the AND plane. The number of rows in the AND plane is three (since there are three inputs), and there are also three rows in the OR plane (since there are three outputs). [Figure A.3.4](#) shows the resulting PLA, with the product terms corresponding to the truth table entries from top to bottom.

read-only memory (ROM) A memory whose contents are designated at creation time, after which the contents can only be read. ROM is used as structured logic to implement a set of logic functions by using the terms in the logic functions as address inputs and the outputs as bits in each word of the memory.

programmable ROM (PROM) A form of read-only memory that can be programmed when a designer knows its contents.

Rather than drawing all the gates, as we do in [Figure A.3.4](#), designers often show just the position of AND gates and OR gates. Dots are used on the intersection of a product term signal line and an input line or an output line when a corresponding AND gate or OR gate is required. [Figure A.3.5](#) shows how the PLA of [Figure A.3.4](#) would look when drawn in this way. The contents of a PLA are fixed when the PLA is created, although there are also forms of PLA-like structures, called *PLAs*, that can be programmed electronically when a designer is ready to use them.

ROMs

Another form of structured logic that can be used to implement a set of logic functions is a **read-only memory (ROM)**. A ROM is called a memory because it has a set of locations that can be read; however, the contents of these locations are fixed, usually at the time the ROM is manufactured. There are also **programmable ROMs (PROMs)** that can be programmed electronically, when a designer knows their contents. There are also erasable PROMs; these devices require a slow erasure process using ultraviolet light, and thus are used as read-only memories, except during the design and debugging process.

A ROM has a set of input address lines and a set of outputs. The number of addressable entries in the ROM determines the number of address lines: if the

ROM contains 2^m addressable entries, called the *height*, then there are m input lines. The number of bits in each addressable entry is equal to the number of output bits and is sometimes called the *width* of the ROM. The total number of bits in the ROM is equal to the height times the width. The height and width are sometimes collectively referred to as the *shape* of the ROM.

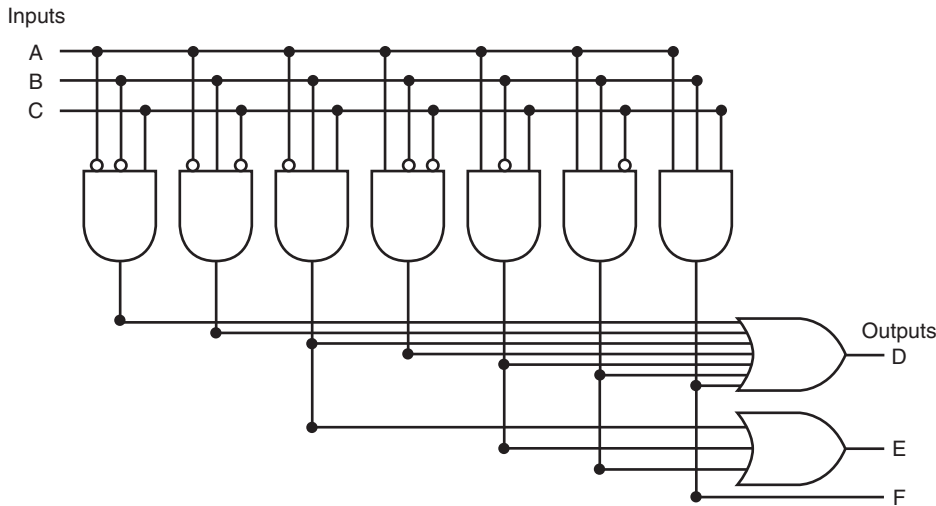


FIGURE A.3.4 The PLA for implementing the logic function described in the example.

A ROM can encode a collection of logic functions directly from the truth table. For example, if there are n functions with m inputs, we need a ROM with m address lines (and 2^m entries), with each entry being n bits wide. The entries in the input portion of the truth table represent the addresses of the entries in the ROM, while the contents of the output portion of the truth table constitute the contents of the ROM. If the truth table is organized so that the sequence of entries in the input portion constitutes a sequence of binary numbers (as have all the truth tables we have shown so far), then the output portion gives the ROM contents in order as well. In the example starting on page A-13, there were three inputs and three outputs. This leads to a ROM with $2^3 = 8$ entries, each 3 bits wide. The contents of those entries in increasing order by address are directly given by the output portion of the truth table that appears on page A-14.

ROMs and PLAs are closely related. A ROM is fully decoded: it contains a full output word for every possible input combination. A PLA is only partially decoded. This means that a ROM will always contain more entries. For the earlier truth table on page A-14, the ROM contains entries for all eight possible inputs, whereas the PLA contains only the seven active product terms. As the number of inputs grows,

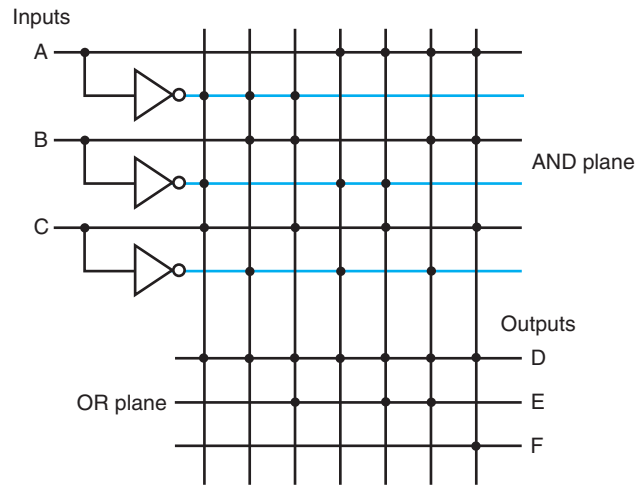


FIGURE A.3.5 A PLA drawn using dots to indicate the components of the product terms and sum terms in the array. Rather than use inverters on the gates, usually all the inputs are run the width of the AND plane in both true and complement forms. A dot in the AND plane indicates that the input, or its inverse, occurs in the product term. A dot in the OR plane indicates that the corresponding product term appears in the corresponding output.

the number of entries in the ROM grows exponentially. In contrast, for most real logic functions, the number of product terms grows much more slowly (see the examples in [Appendix C](#)). This difference makes PLAs generally more efficient for implementing combinational logic functions. ROMs have the advantage of being able to implement any logic function with the matching number of inputs and outputs. This advantage makes it easier to change the ROM contents if the logic function changes, since the size of the ROM need not change.

In addition to ROMs and PLAs, modern logic synthesis systems will also translate small blocks of combinational logic into a collection of gates that can be placed and wired automatically. Although some small collections of gates are usually not area-efficient, for small logic functions they have less overhead than the rigid structure of a ROM and PLA and so are preferred.

For designing logic outside of a custom or semicustom integrated circuit, a common choice is a field programming device; we describe these devices in Section A.12.

Don't Cares

Often in implementing some combinational logic, there are situations where we do not care what the value of some output is, either because another output is true or because a subset of the input combinations determines the values of the outputs. Such situations are referred to as *don't cares*. Don't cares are important because they make it easier to optimize the implementation of a logic function.

There are two types of don't cares: output don't cares and input don't cares, both of which can be represented in a truth table. *Output don't cares* arise when we don't care about the value of an output for some input combination. They appear as Xs in the output portion of a truth table. When an output is a don't care for some input combination, the designer or logic optimization program is free to make the output true or false for that input combination. *Input don't cares* arise when an output depends on only some of the inputs, and they are also shown as Xs, though in the input portion of the truth table.

Don't Cares

Consider a logic function with inputs A , B , and C defined as follows:

- If A or C is true, then output D is true, whatever the value of B .
- If A or B is true, then output E is true, whatever the value of C .
- Output F is true if exactly one of the inputs is true, although we don't care about the value of F , whenever D and E are both true.

Show the full truth table for this function and the truth table using don't cares. How many product terms are required in a PLA for each of these?

Here's the full truth table, without don't cares:

Inputs			Outputs		
A	B	C	D	E	F
0	0	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	1
0	1	1	1	1	0
1	0	0	1	1	1
1	0	1	1	1	0
1	1	0	1	1	0
1	1	1	1	1	0

EXAMPLE

ANSWER

This requires seven product terms without optimization. The truth table written with output don't cares looks like this:

Inputs			Outputs		
A	B	C	D	E	F
0	0	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	1
0	1	1	1	1	X
1	0	0	1	1	X
1	0	1	1	1	X
1	1	0	1	1	X
1	1	1	1	1	X

If we also use the input don't cares, this truth table can be further simplified to yield the following:

Inputs			Outputs		
A	B	C	D	E	F
0	0	0	0	0	0
0	0	1	1	0	1
0	1	0	0	1	1
X	1	1	1	1	X
1	X	X	1	1	X

This simplified truth table requires a PLA with four minterms, or it can be implemented in discrete gates with one two-input AND gate and three OR gates (two with three inputs and one with two inputs). This compares to the original truth table that had seven minterms and would have required four AND gates.

Logic minimization is critical to achieving efficient implementations. One tool useful for hand minimization of random logic is *Karnaugh maps*. Karnaugh maps represent the truth table graphically, so that product terms that may be combined are easily seen. Nevertheless, hand optimization of significant logic functions using Karnaugh maps is impractical, both because of the size of the maps and their complexity. Fortunately, the process of logic minimization is highly mechanical and can be performed by design tools. In the process of minimization, the tools take advantage of the don't cares, so specifying them is important. The textbook references at the end of this appendix provide further discussion on logic minimization, Karnaugh maps, and the theory behind such minimization algorithms.

Arrays of Logic Elements

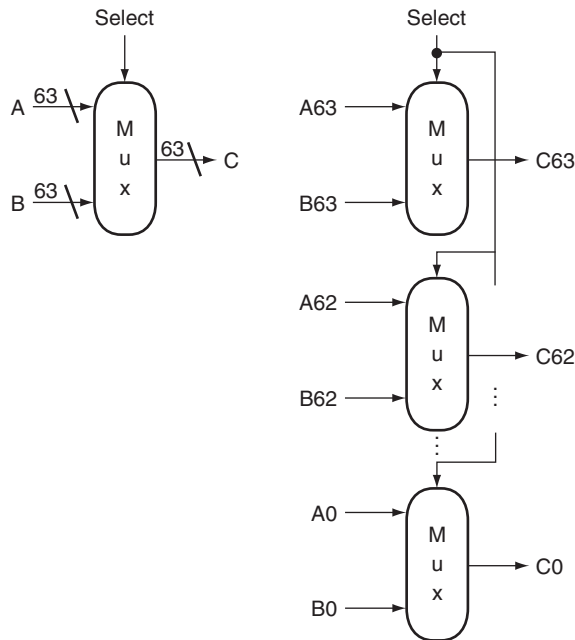
Many of the combinational operations to be performed on data have to be done to an entire word (64 bits) of data. Thus we often want to build an array of logic

elements, which we can represent simply by showing that a given operation will happen to an entire collection of inputs. Inside a machine, much of the time we want to select between a pair of *buses*. A **bus** is a collection of data lines that is treated together as a single logical signal. (The term *bus* is also used to indicate a shared collection of lines with multiple sources and uses.)

For example, in the LEGv8 instruction set, the result of an instruction that is written into a register can come from one of two sources. A multiplexor is used to choose which of the two buses (each 64 bits wide) will be written into the Result register. The 1-bit multiplexor, which we showed earlier, will need to be replicated 64 times.

We indicate that a signal is a bus rather than a single 1-bit line by showing it with a thicker line in a figure. Most buses are 64 bits wide; those that are not are explicitly labeled with their width. When we show a logic unit whose inputs and outputs are buses, this means that the unit must be replicated a sufficient number of times to accommodate the width of the input. Figure A.3.6 shows how we draw a multiplexor that selects between a pair of 64-bit buses and how this expands in terms of 1-bit-wide multiplexors. Sometimes we need to construct an array of logic elements where the inputs for some elements in the array are outputs from earlier elements. For example, this is how a multibit-wide ALU is constructed. In such cases, we must explicitly show how to create wider arrays, since the individual elements of the array are no longer independent, as they are in the case of a 64-bit-wide multiplexor.

bus In logic design, a collection of data lines that is treated together as a single logical signal; also, a shared collection of lines with multiple sources and uses.



a. A 64-bit wide 2-to-1 multiplexor

b. The 64-bit wide multiplexor is actually an array of 64 1-bit multiplexors

FIGURE A.3.6 A multiplexor is arrayed 64 times to perform a selection between two 64-bit inputs. Note that there is still only one data selection signal used for all 64 1-bit multiplexors.

Check Yourself

Parity is a function in which the output depends on the number of 1s in the input. For an even parity function, the output is 1 if the input has an even number of ones. Suppose a ROM is used to implement an even parity function with a 4-bit input. Which of A, B, C, or D represents the contents of the ROM?

Address	A	B	C	D
0	0	1	0	1
1	0	1	1	0
2	0	1	0	1
3	0	1	1	0
4	0	1	0	1
5	0	1	1	0
6	0	1	0	1
7	0	1	1	0
8	1	0	0	1
9	1	0	1	0
10	1	0	0	1
11	1	0	1	0
12	1	0	0	1
13	1	0	1	0
14	1	0	0	1
15	1	0	1	0

A.4

Using a Hardware Description Language

hardware description language

A programming language for describing hardware, used for generating simulations of a hardware design and also as input to synthesis tools that can generate actual hardware.

Verilog One of the two most common hardware description languages.

VHDL One of the two most common hardware description languages.

Today most digital design of processors and related hardware systems is done using a **hardware description language**. Such a language serves two purposes. First, it provides an abstract description of the hardware to simulate and debug the design. Second, with the use of logic synthesis and hardware compilation tools, this description can be compiled into the hardware implementation.

In this section, we introduce the hardware description language Verilog and show how it can be used for combinational design. In the rest of the appendix, we expand the use of Verilog to include design of sequential logic. In the optional sections of [Chapter 4](#) that appear online, we use Verilog to describe processor implementations. In the optional section from [Chapter 5](#) that appears online, we use system Verilog to describe cache controller implementations. System Verilog adds structures and some other useful features to Verilog.

Verilog is one of the two primary hardware description languages; the other is **VHDL**. Verilog is somewhat more heavily used in industry and is based on C, as opposed to VHDL, which is based on Ada. The reader generally familiar with C will find the basics of Verilog, which we use in this appendix, easy to follow.

Readers already familiar with VHDL should find the concepts simple, provided they have been exposed to the syntax of C.

Verilog can specify both a behavioral and a structural definition of a digital system. A **behavioral specification** describes how a digital system functionally operates. A **structural specification** describes the detailed organization of a digital system, usually using a hierarchical description. A structural specification can be used to describe a hardware system in terms of a hierarchy of basic elements such as gates and switches. Thus, we could use Verilog to describe the exact contents of the truth tables and datapath of the last section.

With the arrival of **hardware synthesis tools**, most designers now use Verilog or VHDL to structurally describe only the datapath, relying on logic synthesis to generate the control from a behavioral description. In addition, most CAD systems provide extensive libraries of standardized parts, such as ALUs, multiplexors, register files, memories, and programmable logic blocks, as well as basic gates.

Obtaining an acceptable result using libraries and logic synthesis requires that the specification be written with an eye toward the eventual synthesis and the desired outcome. For our simple designs, this primarily means making clear what we expect to be implemented in combinational logic and what we expect to require in sequential logic. In most of the examples we use in this section and the remainder of this appendix, we have written the Verilog with the eventual synthesis in mind.

behavioral specification

Describes how a digital system operates functionally.

structural specification

Describes how a digital system is organized in terms of a hierarchical connection of elements.

hardware synthesis tools

Computer-aided design software that can generate a gate-level design based on behavioral descriptions of a digital system.

Datatypes and Operators in Verilog

There are two primary datatypes in Verilog:

1. A **wire** specifies a combinational signal.
2. A **reg** (register) holds a value, which can vary with time. A reg need not necessarily correspond to an actual register in an implementation, although it often will.

wire In Verilog, specifies a combinational signal.

reg In Verilog, a register.

A register or wire, named X, that is 64 bits wide is declared as an array: `reg [63:0] X` or `wire [63:0] X`, which also sets the index of 0 to designate the least significant bit of the register. Because we often want to access a subfield of a register or wire, we can refer to a contiguous set of bits of a register or wire with the notation `[starting bit: ending bit]`, where both indices must be constant values.

An array of registers is used for a structure like a register file or memory. Thus, the declaration

```
reg [63:0] registerfile[0:31]
```

specifies a variable `registerfile` that is equivalent to a LEGv8 registerfile, where register 0 is the first. When accessing an array, we can refer to a single element, as in C, using the notation `registerfile[regnum]`.

The possible values for a register or wire in Verilog are

- 0 or 1, representing logical false or true
- X, representing unknown, the initial value given to all registers and to any wire not connected to something
- Z, representing the high-impedance state for tristate gates, which we will not discuss in this appendix

Constant values can be specified as decimal numbers as well as binary, octal, or hexadecimal. We often want to say exactly how large a constant field is in bits. This is done by prefixing the value with a decimal number specifying its size in bits. For example:

- `4'b0100` specifies a 4-bit binary constant with the value 4, as does `4'd4`.
- `-8'h4` specifies an 8-bit constant with the value -4 (in two's complement representation)

Values can also be concatenated by placing them within `{ }` separated by commas. The notation `{x{bitfield}}` replicates `bitfield` `x` times. For example:

- `{32{2'b01}}` creates a 64-bit value with the pattern 0101 ... 01.
- `{A[31:16],B[15:0]}` creates a value whose upper 16 bits come from A and whose lower 16 bits come from B.

Verilog provides the full set of unary and binary operators from C, including the arithmetic operators (+, -, *, /), the logical operators (&, |, ~), the comparison operators (=, !=, >, <, <=, >=), the shift operators (<<, >>), and C's conditional operator (?), which is used in the form `condition ? expr1 : expr2` and returns `expr1` if the condition is true and `expr2` if it is false). Verilog adds a set of unary logic reduction operators (&, |, ^) that yield a single bit by applying the logical operator to all the bits of an operand. For example, `&A` returns the value obtained by ANDing all the bits of A together, and `^A` returns the reduction obtained by using exclusive OR on all the bits of A.

Check Yourself

Which of the following define exactly the same value?

1. `8'bimoooo`
2. `8'hF0`
3. `8'd240`
4. `{{4{1'b1}}, {4{1'b0}}}`
5. `{4'b1,4'b0}`

Structure of a Verilog Program

A Verilog program is structured as a set of modules, which may represent anything from a collection of logic gates to a complete system. Modules are similar to classes in C++, although not nearly as powerful. A module specifies its input and output ports, which describe the incoming and outgoing connections of a module. A module may also declare additional variables. The body of a module consists of:

- `initial` constructs, which can initialize `reg` variables
- Continuous assignments, which define only combinational logic
- `always` constructs, which can define either sequential or combinational logic
- Instances of other modules, which are used to implement the module being defined

Representing Complex Combinational Logic in Verilog

A continuous assignment, which is indicated with the keyword `assign`, acts like a combinational logic function: the output is continuously assigned the value, and a change in the input values is reflected immediately in the output value. Wires may only be assigned values with continuous assignments. Using continuous assignments, we can define a module that implements a half-adder, as [Figure A.4.1](#) shows.

Assign statements are one sure way to write Verilog that generates combinational logic. For more complex structures, however, assign statements may be awkward or tedious to use. It is also possible to use the `always` block of a module to describe a combinational logic element, although care must be taken. Using an `always` block allows the inclusion of Verilog control constructs, such as *if-then-else*, *case* statements, *for* statements, and *repeat* statements, to be used. These statements are similar to those in C with small changes.

An `always` block specifies an optional list of signals on which the block is sensitive (in a list starting with `@`). The `always` block is re-evaluated if any of the

```
module half_adder (A,B,Sum,Carry);
    input A,B; //two 1-bit inputs
    output Sum, Carry; //two 1-bit outputs
    assign Sum = A ^ B; //sum is A xor B
    assign Carry = A & B; //Carry is A and B
endmodule
```

FIGURE A.4.1 A Verilog module that defines a half-adder using continuous assignments.

sensitivity list The list of signals that specifies when an `always` block should be re-evaluated.

blocking assignment In Verilog, an assignment that completes before the execution of the next statement.

nonblocking assignment An assignment that continues after evaluating the right-hand side, assigning the left-hand side the value only after all right-hand sides are evaluated.

listed signals changes value; if the list is omitted, the `always` block is constantly re-evaluated. When an `always` block is specifying combinational logic, the **sensitivity list** should include all the input signals. If there are multiple Verilog statements to be executed in an `always` block, they are surrounded by the keywords `begin` and `end`, which take the place of the `{` and `}` in C. An `always` block thus looks like this:

```
always @(list of signals that cause reevaluation) begin
    Verilog statements including assignments and other
control statements end
```

Reg variables may only be assigned inside an `always` block, using a procedural assignment statement (as distinguished from continuous assignment we saw earlier). There are, however, two different types of procedural assignments. The assignment operator `=` executes as it does in C; the right-hand side is evaluated, and the left-hand side is assigned the value. Furthermore, it executes like the normal C assignment statement: that is, it is completed before the next statement is executed. Hence, the assignment operator `=` has the name **blocking assignment**. This blocking can be useful in the generation of sequential logic, and we will return to it shortly. The other form of assignment (**nonblocking**) is indicated by `<=`. In nonblocking assignment, all right-hand sides of the assignments in an `always` group are evaluated and the assignments are done simultaneously. As a first example of combinational logic implemented using an `always` block, [Figure A.4.2](#) shows the implementation of a 4-to-1 multiplexor, which uses a `case` construct to make it easy to write. The `case` construct looks like a C `switch` statement. [Figure A.4.3](#) shows a definition of a LEGv8 ALU, which also uses a `case` statement.

Since only reg variables may be assigned inside `always` blocks, when we want to describe combinational logic using an `always` block, care must be taken to ensure that the reg does not synthesize into a register. A variety of pitfalls are described in the elaboration below.

Elaboration: Continuous assignment statements always yield combinational logic, but other Verilog structures, even when in `always` blocks, can yield unexpected results during logic synthesis. The most common problem is creating sequential logic by implying the existence of a latch or register, which results in an implementation that is both slower and more costly than perhaps intended. To ensure that the logic that you intend to be combinational is synthesized that way, make sure you do the following:

1. Place all combinational logic in a continuous assignment or an `always` block.
2. Make sure that all the signals used as inputs appear in the sensitivity list of an `always` block.
3. Ensure that every path through an `always` block assigns a value to the exact same set of bits.

The last of these is the easiest to overlook; read through the example in [Figure A.5.15](#) to convince yourself that this property is adhered to.

```

module Mult4to1 (In1,In2,In3,In4,Sel,Out);
    input [31:0] In1, In2, In3, In4; //four 32-bit inputs
    input [1:0] Sel; //selector signal
    output reg [31:0] Out; // 32-bit output
    always @(In1, In2, In3, In4, Sel)
        case (Sel) //a 4->1 multiplexor
            0: Out <= In1;
            1: Out <= In2;
            2: Out <= In3;
            default: Out <= In4;
        endcase
endmodule

```

FIGURE A.4.2 A Verilog definition of a 4-to-1 multiplexor with 32-bit inputs, using a `case` statement. The `case` statement acts like a `C switch` statement, except that in Verilog only the code associated with the selected case is executed (as if each case state had a `break` at the end) and there is no fall-through to the next statement.

```

module MIPSALU (ALUctl, A, B, ALUOut, Zero);
    input [3:0] ALUctl;
    input [31:0] A,B;
    output reg [31:0] ALUOut;
    output Zero;
    assign Zero = (ALUOut==0); //Zero is true if ALUOut is 0; goes anywhere
    always @(ALUctl, A, B) //reevaluate if these change
        case (ALUctl)
            0: ALUOut <= A & B;
            1: ALUOut <= A | B;
            2: ALUOut <= A + B;
            6: ALUOut <= A - B;
            7: ALUOut <= A < B ? 1:0;
            12: ALUOut <= ~(A | B); // result is nor
            default: ALUOut <= 0; //default to 0, should not happen;
        endcase
endmodule

```

FIGURE A.4.3 A Verilog behavioral definition of a LEGv8 ALU. This could be synthesized using a module library containing basic arithmetic and logical operations.

Check Yourself

Assuming all values are initially zero, what are the values of A and B after executing this Verilog code inside an `always` block?

```
C = 1;
A <= C;
B = C;
```

A.5**Constructing a Basic Arithmetic Logic Unit**

ALU n. [Arthritic Logic Unit or (rare) Arithmetic Logic Unit] A random-number generator supplied as standard with all computer systems.

Stan Kelly-Bootle, *The Devil's DP Dictionary*, 1981

The **arithmetic logic unit (ALU)** is the brawn of the computer, the device that performs the arithmetic operations like addition and subtraction or logical operations like AND and OR. This section constructs an ALU from four hardware building blocks (AND and OR gates, inverters, and multiplexors) and illustrates how combinational logic works. In the next section, we will see how addition can be sped up through more clever designs.

Because the LEGv8 word is 64 bits wide, we need a 64-bit-wide ALU. Let's assume that we will connect 64 1-bit ALUs to create the desired ALU. We'll therefore start by constructing a 1-bit ALU.

A 1-Bit ALU

The logical operations are easiest, because they map directly onto the hardware components in [Figure A.2.1](#).

The 1-bit logical unit for AND and OR looks like [Figure A.5.1](#). The multiplexor on the right then selects *a* AND *b* or *a* OR *b*, depending on whether the value of *Operation* is 0 or 1. The line that controls the multiplexor is shown in color to distinguish it from the lines containing data. Notice that we have renamed the control and output lines of the multiplexor to give them names that reflect the function of the ALU.

The next function to include is addition. An adder must have two inputs for the operands and a single-bit output for the sum. There must be a second output to pass on the carry, called *CarryOut*. Since the *CarryOut* from the neighbor adder must be included as an input, we need a third input. This input is called *CarryIn*. [Figure A.5.2](#) shows the inputs and the outputs of a 1-bit adder. Since we know what addition is supposed to do, we can specify the outputs of this “black box” based on its inputs, as [Figure A.5.3](#) demonstrates.

We can express the output functions *CarryOut* and *Sum* as logical equations, and these equations can in turn be implemented with logic gates. Let's do *CarryOut*. [Figure A.5.4](#) shows the values of the inputs when *CarryOut* is a 1.

We can turn this truth table into a logical equation:

$$\text{CarryOut} = (b \cdot \text{CarryIn}) + (a \cdot \text{CarryIn}) + (a \cdot b) + (a \cdot b \cdot \text{CarryIn})$$

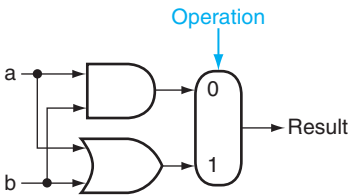


FIGURE A.5.1 The 1-bit logical unit for AND and OR.

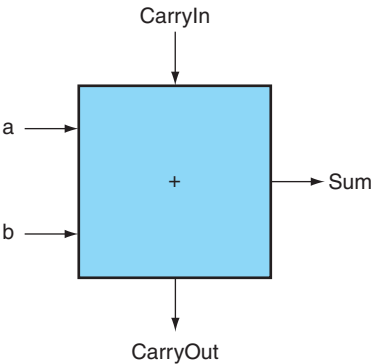


FIGURE A.5.2 A 1-bit adder. This adder is called a full adder; it is also called a (3,2) adder because it has three inputs and two outputs. An adder with only the a and b inputs is called a (2,2) adder or half-adder.

Inputs			Outputs		Comments
a	b	CarryIn	CarryOut	Sum	
0	0	0	0	0	$0 + 0 + 0 = 00_{\text{two}}$
0	0	1	0	1	$0 + 0 + 1 = 01_{\text{two}}$
0	1	0	0	1	$0 + 1 + 0 = 01_{\text{two}}$
0	1	1	1	0	$0 + 1 + 1 = 10_{\text{two}}$
1	0	0	0	1	$1 + 0 + 0 = 01_{\text{two}}$
1	0	1	1	0	$1 + 0 + 1 = 10_{\text{two}}$
1	1	0	1	0	$1 + 1 + 0 = 10_{\text{two}}$
1	1	1	1	1	$1 + 1 + 1 = 11_{\text{two}}$

FIGURE A.5.3 Input and output specification for a 1-bit adder.

If $a \cdot b \cdot \text{CarryIn}$ is true, then all of the other three terms must also be true, so we can leave out this last term corresponding to the fourth line of the table. We can thus simplify the equation to

$$\text{CarryOut} = (b \cdot \text{CarryIn}) + (a \cdot \text{CarryIn}) + (a \cdot b)$$

Figure A.5.5 shows that the hardware within the adder black box for CarryOut consists of three AND gates and one OR gate. The three AND gates correspond exactly to the three parenthesized terms of the formula above for CarryOut, and the OR gate sums the three terms.

Inputs		
a	b	CarryIn
0	1	1
1	0	1
1	1	0
1	1	1

FIGURE A.5.4 Values of the inputs when CarryOut is a 1.

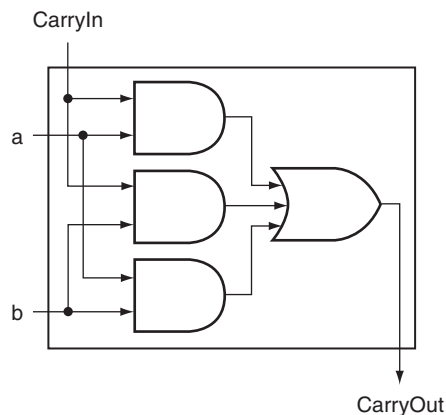


FIGURE A.5.5 Adder hardware for the CarryOut signal. The rest of the adder hardware is the logic for the Sum output given in the equation on this page.

The Sum bit is set when exactly one input is 1 or when all three inputs are 1. The Sum results in a complex Boolean equation (recall that $\bar{a}$ means NOT a):

$$\text{Sum} = (a \cdot \bar{b} \cdot \overline{\text{CarryIn}}) + (\bar{a} \cdot b \cdot \overline{\text{CarryIn}}) + (\bar{a} \cdot \bar{b} \cdot \text{CarryIn}) + (a \cdot b \cdot \text{CarryIn})$$

The drawing of the logic for the Sum bit in the adder black box is left as an exercise for the reader.

Figure A.5.6 shows a 1-bit ALU derived by combining the adder with the earlier components. Sometimes designers also want the ALU to perform a few more simple operations, such as generating 0. The easiest way to add an operation is to expand the multiplexor controlled by the Operation line and, for this example, to connect 0 directly to the new input of that expanded multiplexor.

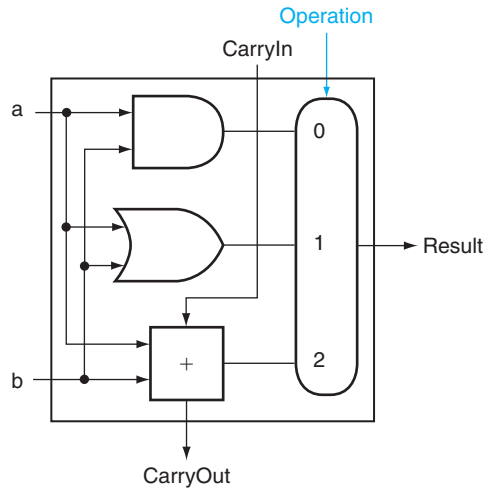


FIGURE A.5.6 A 1-bit ALU that performs AND, OR, and addition (see Figure A.5.5).

A 64-Bit ALU

Now that we have completed the 1-bit ALU, the full 64-bit ALU is created by connecting adjacent “black boxes.” Using x_i to mean the i th bit of x , Figure A.5.7 shows a 64-bit ALU. Just as a single stone can cause ripples to radiate to the shores of a quiet lake, a single carry out of the least significant bit (Result0) can ripple all the way through the adder, causing a carry out of the most significant bit (Result31). Hence, the adder created by directly linking the carries of 1-bit adders is called a *ripple carry* adder. We’ll see a faster way to connect the 1-bit adders starting on page A-38.

Subtraction is the same as adding the negative version of an operand, and this is how adders perform subtraction. Recall that the shortcut for negating a two’s complement number is to invert each bit (sometimes called the *one’s complement*) and then add 1. To invert each bit, we simply add a 2:1 multiplexor that chooses between b and $\bar{b}$, as Figure A.5.8 shows.

Suppose we connect 64 of these 1-bit ALUs, as we did in Figure A.5.7. The added multiplexor gives the option of b or its inverted value, depending on B_{invert} , but

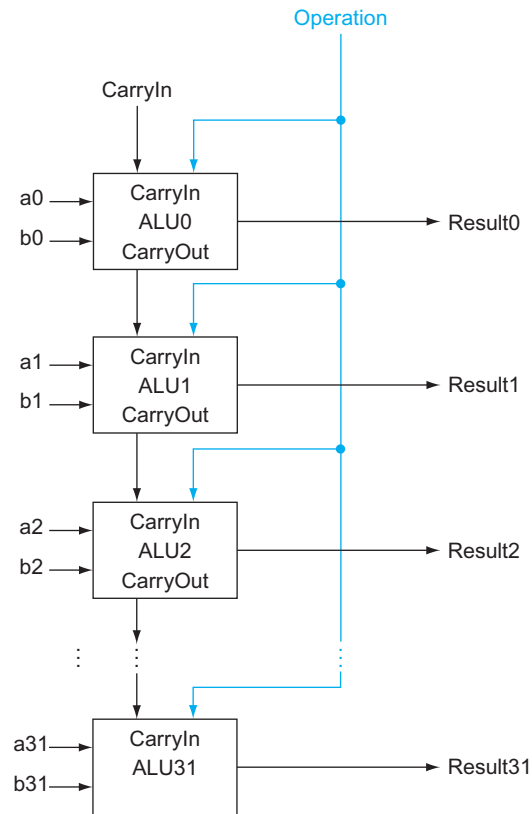


FIGURE A.5.7 A 64-bit ALU constructed from 64 1-bit ALUs. CarryOut of the less significant bit is connected to the CarryIn of the more significant bit. This organization is called ripple carry.

this is only one step in negating a two's complement number. Notice that the least significant bit still has a CarryIn signal, even though it's unnecessary for addition. What happens if we set this CarryIn to 1 instead of 0? The adder will then calculate $a + b + 1$. By selecting the inverted version of b , we get exactly what we want:

$$a + \bar{b} + 1 = a + (\bar{b} + 1) = a + (-b) = a - b$$

The simplicity of the hardware design of a two's complement adder helps explain why two's complement representation has become the universal standard for integer computer arithmetic.

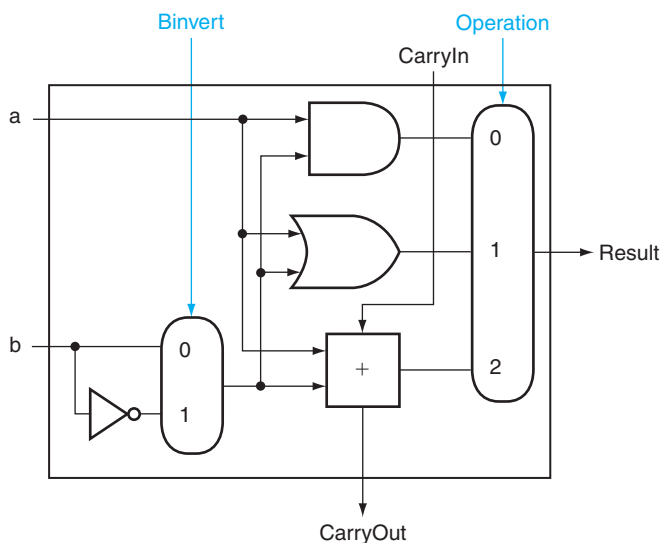


FIGURE A.5.8 A 1-bit ALU that performs AND, OR, and addition on a and b or a and $\bar{b}$. By selecting $\bar{b}$ (Binvert = 1) and setting CarryIn to 1 in the least significant bit of the ALU, we get two's complement subtraction of b from a instead of addition of b to a .

A LEGv8 ALU also needs a NOR function. Instead of adding a separate gate for NOR, we can reuse much of the hardware already in the ALU, like we did for subtract. The insight comes from the following truth about NOR:

$$\overline{(a + b)} = \bar{a} \cdot \bar{b}$$

That is, NOT (a OR b) is equivalent to NOT a AND NOT b . This fact is called DeMorgan's theorem and is explored in the exercises in more depth.

Since we have AND and NOT b , we only need to add NOT a to the ALU. [Figure A.5.9](#) shows that change.

Tailoring the 64-Bit ALU to LEGv8

These four operations—add, subtract, AND, OR—are found in the ALU of almost every computer, and the operations of most LEGv8 instructions can be performed by this ALU. But the design of the ALU is incomplete.

One instruction that still needs support is the compare and branch zero (CBZ). Recall that the operation the ALU just passes the register input value. For the ALU to perform CBZ, we first need to expand the three-input multiplexor in [Figure A.5.8](#) to add an input for the CBZ result. We call that new input Pass and use it only for CBZ.

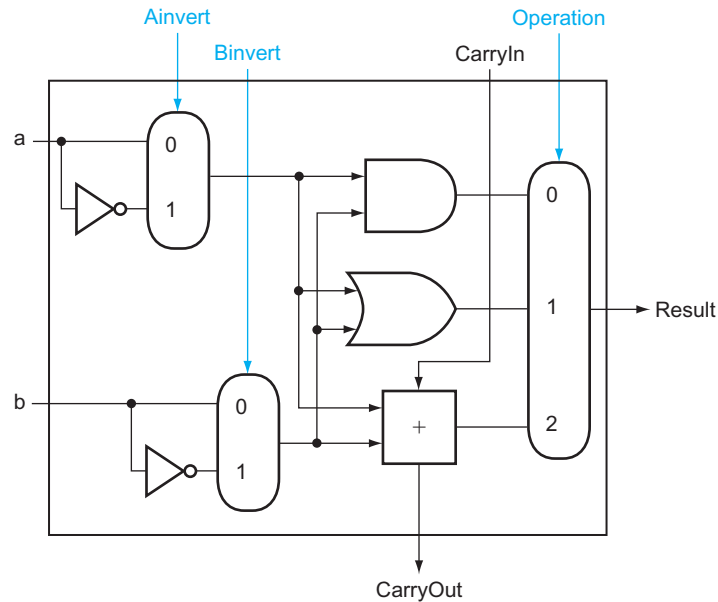


FIGURE A.5.9 A 1-bit ALU that performs AND, OR, and addition on a and b or $\bar{a}$ and $\bar{b}$. By selecting $\bar{a}$ ($Ainvert = 1$) and $\bar{b}$ ($Binvert = 1$), we get a NOR b instead of a AND b .

The top drawing of [Figure A.5.10](#) shows the new 1-bit ALU with the expanded multiplexor. [Figure A.5.11](#) shows the 64-bit ALU.

To further tailor the ALU to the LEGv8 instruction set, we must support conditional branch instructions. These instructions branch either if two registers are equal or if they are unequal. The easiest way to test equality with the ALU is to subtract b from a and then test to see if the result is 0, since

$$(a - b = 0) \Rightarrow a = b$$

Thus, if we add hardware to test if the result is 0, we can test for equality. The simplest way is to OR all the outputs together and then send that signal through an inverter:

$$\text{Zero} = \overline{(\text{Result}_{63} + \text{Result}_{62} + \dots + \text{Result}_2 + \text{Result}_1 + \text{Result}_0)}$$

[Figure A.5.12](#) shows the revised 64-bit ALU. We can think of the combination of the 1-bit $Ainvert$ line, the 1-bit $Binvert$ line, and the 2-bit $Operation$ lines as 4-bit control lines for the ALU, telling it to perform add, subtract, AND, OR, or set on less than. [Figure A.5.13](#) shows the ALU control lines and the corresponding ALU operation.

Finally, now that we have seen what is inside a 64-bit ALU, we will use the universal symbol for a complete ALU, as shown in [Figure A.5.14](#).

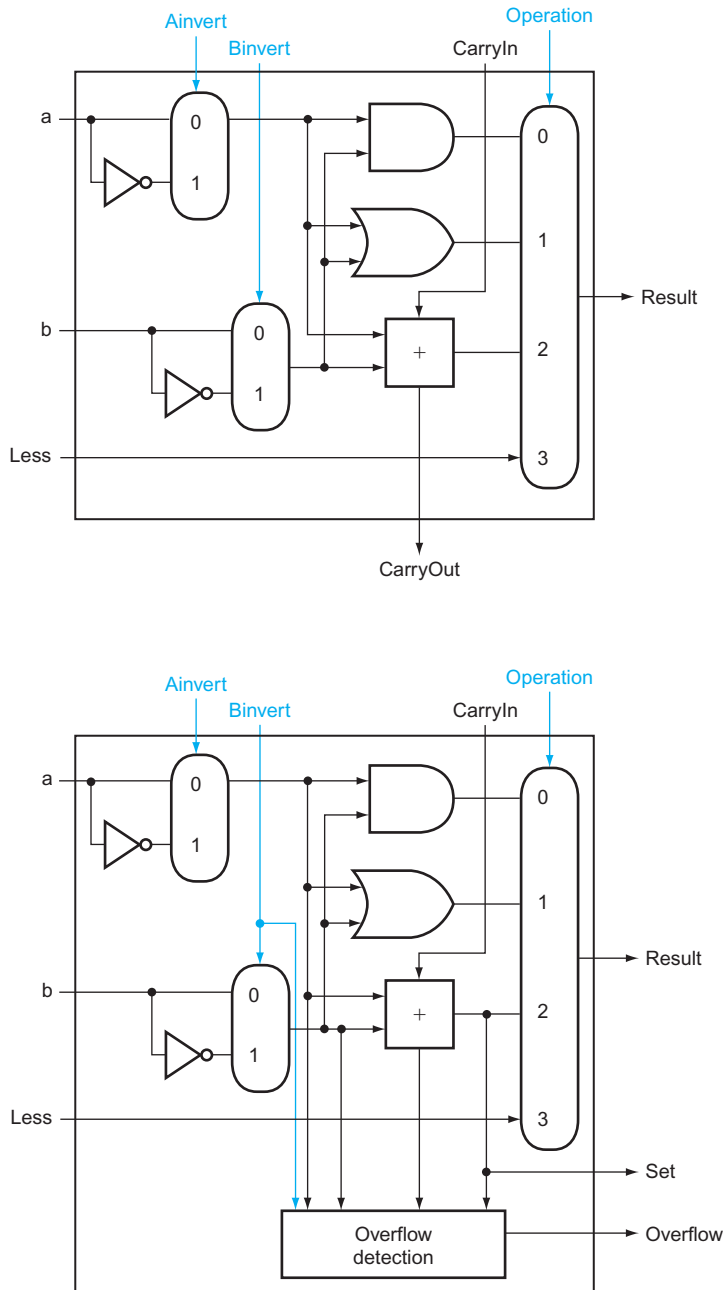


FIGURE A.5.10 (Top) A 1-bit ALU that performs AND, OR, and addition on *a* and *b* or $\bar{b}$, and (bottom) a 1-bit ALU for the most significant bit. The top drawing includes a direct input that is connected to perform the set on less than operation (see Figure A.5.11); the bottom has a direct output from the adder for the less than comparison called Set. (See Exercise A.24 at the end of this appendix to see how to calculate overflow with fewer inputs.)

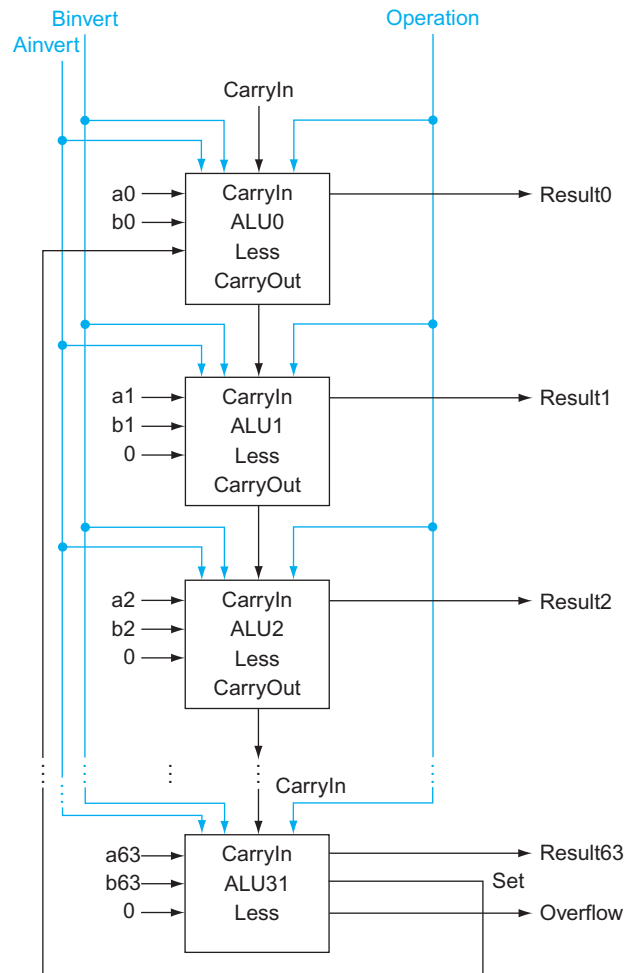


FIGURE A.5.11 A 64-bit ALU constructed from the 63 copies of the 1-bit ALU in the top of Figure A.5.10 and one 1-bit ALU in the bottom of that figure. The Less inputs are connected to 0 except for the least significant bit, which is connected to the Set output of the most significant bit. If the ALU performs $a - b$ and we select the input 3 in the multiplexor in Figure A.5.10, then Result = 0 ... 001 if $a < b$, and Result = 0 ... 000 otherwise.

Defining the LEGv8 ALU in Verilog

Figure A.5.15 shows how a combinational LEGv8 ALU might be specified in Verilog; such a specification would probably be compiled using a standard parts library that provided an adder, which could be instantiated. For completeness, we show the ALU control for LEGv8 in Figure A.5.16, which is used in Chapter 4, where we build a Verilog version of the LEGv8 datapath.

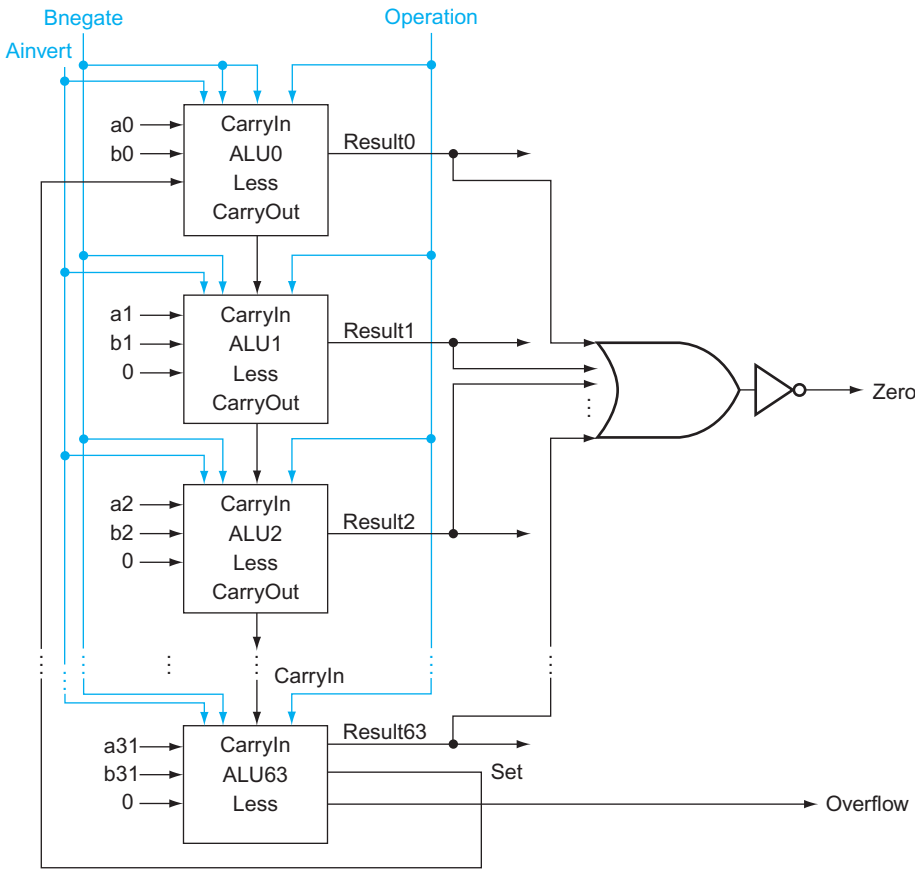


FIGURE A.5.12 The final 64-bit ALU. This adds a Zero detector to Figure A.5.11.

ALU control lines	Function
0000	AND
0001	OR
0010	add
0110	subtract
0111	set on less than
1100	NOR

FIGURE A.5.13 The values of the three ALU control lines, $Bnegate$, and $Operation$, and the corresponding ALU operations.

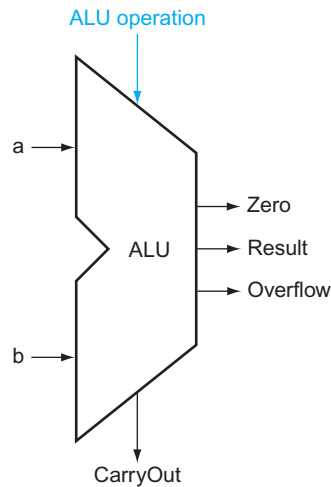


FIGURE A.5.14 The symbol commonly used to represent an ALU, as shown in Figure A.5.12. This symbol is also used to represent an adder, so it is normally labeled either with ALU or Adder.

```

module MIPSALU (ALUctl, A, B, ALUOut, Zero);
    input [3:0] ALUctl;
    input [63:0] A,B;
    output reg [63:0] ALUOut;
    output Zero;
    assign Zero = (ALUOut==0); //Zero is true if ALUOut is 0
    always @(ALUctl, A, B) begin //reevaluate if these change
        case (ALUctl)
            0: ALUOut <= A & B;
            1: ALUOut <= A | B;
            2: ALUOut <= A + B;
            6: ALUOut <= A - B;
            7: ALUOut <= A < B ? 1 : 0;
            12: ALUOut <= ~(A | B); // result is nor
            default: ALUOut <= 0;
        endcase
    end
endmodule

```

FIGURE A.5.15 A Verilog behavioral definition of a LEGv8 ALU.

```

module ALUControl (ALUOp, FuncCode, ALUCtl);
    input [1:0] ALUOp;
    input [5:0] FuncCode;
    output [3:0] reg ALUCtl;
    always case (FuncCode)
        32: ALUOp<=2; // add
        34: ALUOp<=6; //subtract
        36: ALUOp<=0; // and
        37: ALUOp<=1; // or
        39: ALUOp<=12; // nor
        42: ALUOp<=7; // slt
        default: ALUOp<=15; // should not happen
    endcase
endmodule

```

FIGURE A.5.16 The LEGv8 ALU control: a simple piece of combinational control logic.

The next question is, “How quickly can this ALU add two 64-bit operands?” We can determine the *a* and *b* inputs, but the CarryIn input depends on the operation in the adjacent 1-bit adder. If we trace all the way through the chain of dependencies, we connect the most significant bit to the least significant bit, so the most significant bit of the sum must wait for the *sequential* evaluation of all 64 1-bit adders. This sequential chain reaction is too slow to be used in time-critical hardware. The next section explores how to speed-up addition. This topic is not crucial to understanding the rest of the appendix and may be skipped.

Suppose you wanted to add the operation NOT (*a* AND *b*), called NAND. How could the ALU change to support it?

**Check
Yourself**

1. **No change.** You can calculate NAND quickly using the current ALU since $(a \cdot b) = \overline{a + b}$ and we already have NOT *a*, NOT *b*, and OR.
2. You must expand the big multiplexor to add another input, and then add new logic to calculate NAND.

A.6

Faster Addition: Carry Lookahead

The key to speeding up addition is determining the carry in to the high-order bits sooner. There are a variety of schemes to anticipate the carry so that the worst-case scenario is a function of the $\log_2$ of the number of bits in the adder. These

anticipatory signals are faster because they go through fewer gates in sequence, but it takes many more gates to anticipate the proper carry.

A key to understanding fast-carry schemes is to remember that, unlike software, hardware executes in parallel whenever inputs change.

Fast Carry Using “Infinite” Hardware

As we mentioned earlier, any equation can be represented in two levels of logic. Since the only external inputs are the two operands and the CarryIn to the least significant bit of the adder, in theory we could calculate the CarryIn values to all the remaining bits of the adder in just two levels of logic.

For example, the CarryIn for bit 2 of the adder is exactly the CarryOut of bit 1, so the formula is

$$\text{CarryIn}_2 = (b_1 \cdot \text{CarryIn}_1) + (a_1 \cdot \text{CarryIn}_1) + (a_1 \cdot b_1)$$

Similarly, CarryIn1 is defined as

$$\text{CarryIn}_1 = (b_0 \cdot \text{CarryIn}_0) + (a_0 \cdot \text{CarryIn}_0) + (a_0 \cdot b_0)$$

Using the shorter and more traditional abbreviation of c_i for CarryIn $_i$, we can rewrite the formulas as

$$c_2 = (b_1 \cdot c_1) + (a_1 \cdot c_1) + (a_1 \cdot b_1)$$

$$c_1 = (b_0 \cdot c_0) + (a_0 \cdot c_0) + (a_0 \cdot b_0)$$

Substituting the definition of c_1 for the first equation results in this formula:

$$\begin{aligned} c_2 = & (a_1 \cdot a_0 \cdot b_0) + (a_1 \cdot a_0 \cdot c_0) + (a_1 \cdot b_0 \cdot c_0) \\ & + (b_1 \cdot a_0 \cdot b_0) + (b_1 \cdot a_0 \cdot c_0) + (b_1 \cdot b_0 \cdot c_0) + (a_1 \cdot b_1) \end{aligned}$$

You can imagine how the equation expands as we get to higher bits in the adder; it grows rapidly with the number of bits. This complexity is reflected in the cost of the hardware for fast carry, making this simple scheme prohibitively expensive for wide adders.

Fast Carry Using the First Level of Abstraction: Propagate and Generate

Most fast-carry schemes limit the complexity of the equations to simplify the hardware, while still making substantial speed improvements over ripple carry. One such scheme is a *carry-lookahead adder*. In [Chapter 1](#), we said computer systems cope with complexity by using levels of abstraction. A carry-lookahead adder relies on levels of abstraction in its implementation.

Let's factor our original equation as a first step:

$$\begin{aligned} ci + 1 &= (bi \cdot ci) + (ai \cdot ci) + (ai \cdot bi) \\ &= (ai \cdot bi) + (ai + bi) \cdot ci \end{aligned}$$

If we were to rewrite the equation for c_2 using this formula, we would see some repeated patterns:

$$c_2 = (a_1 \cdot b_1) + (a_1 \cdot b_1) \cdot ((a_0 \cdot b_0) + (a_0 + b_0) \cdot c_0)$$

Note the repeated appearance of $(ai \cdot bi)$ and $(ai + bi)$ in the formula above. These two important factors are traditionally called *generate* (gi) and *propagate* (pi):

$$\begin{aligned} gi &= ai \cdot bi \\ pi &= ai + bi \end{aligned}$$

Using them to define $ci + 1$, we get

$$ci + 1 = gi + pi \cdot ci$$

To see where the signals get their names, suppose gi is 1. Then

$$ci + 1 = gi + pi \cdot ci = 1 + pi \cdot ci = 1$$

That is, the adder *generates* a CarryOut ($ci + 1$) independent of the value of CarryIn (ci). Now suppose that gi is 0 and pi is 1. Then

$$ci + 1 = gi + pi \cdot ci = 0 + 1 \cdot ci = ci$$

That is, the adder *propagates* CarryIn to a CarryOut. Putting the two together, CarryIn $i + 1$ is a 1 if either gi is 1 or both pi is 1 and CarryIn i is 1.

As an analogy, imagine a row of dominoes set on edge. The end domino can be tipped over by pushing one far away, provided there are no gaps between the two. Similarly, a carry out can be made true by a generate far away, provided all the propagates between them are true.

Relying on the definitions of propagate and generate as our first level of abstraction, we can express the CarryIn signals more economically. Let's show it for 4 bits:

$$\begin{aligned} c_1 &= g_0 + (p_0 \cdot c_0) \\ c_2 &= g_1 + (p_1 \cdot g_0) + (p_1 \cdot p_0 \cdot c_0) \\ c_3 &= g_2 + (p_2 \cdot g_1) + (p_2 \cdot p_1 \cdot g_0) + (p_2 \cdot p_1 \cdot p_0 \cdot c_0) \\ c_4 &= g_3 + (p_3 \cdot g_2) + (p_3 \cdot p_2 \cdot g_1) + (p_3 \cdot p_2 \cdot p_1 \cdot g_0) \\ &\quad + (p_3 \cdot p_2 \cdot p_1 \cdot p_0 \cdot c_0) \end{aligned}$$

These equations just represent common sense: CarryIn_i is a 1 if some earlier adder generates a carry and all intermediary adders propagate a carry. Figure A.6.1 uses plumbing to try to explain carry lookahead.

Even this simplified form leads to large equations and, hence, considerable logic even for a 16-bit adder. Let's try moving to two levels of abstraction.

Fast Carry Using the Second Level of Abstraction

First, we consider this 4-bit adder with its carry-lookahead logic as a single building block. If we connect them in ripple carry fashion to form a 16-bit adder, the add will be faster than the original with a little more hardware.

To go faster, we'll need carry lookahead at a higher level. To perform carry lookahead for 4-bit adders, we need to propagate and generate signals at this higher level. Here they are for the four 4-bit adder blocks:

$$\begin{aligned} P_0 &= p_3 \cdot p_2 \cdot p_1 \cdot p_0 \\ P_1 &= p_7 \cdot p_6 \cdot p_5 \cdot p_4 \\ P_2 &= p_{11} \cdot p_{10} \cdot p_9 \cdot p_8 \\ P_3 &= p_{15} \cdot p_{14} \cdot p_{13} \cdot p_{12} \end{aligned}$$

That is, the "super" propagate signal for the 4-bit abstraction (P_i) is true only if each of the bits in the group will propagate a carry.

For the "super" generate signal (G_i), we care only if there is a carry out of the most significant bit of the 4-bit group. This obviously occurs if generate is true for that most significant bit; it also occurs if an earlier generate is true *and* all the intermediate propagates, including that of the most significant bit, are also true:

$$\begin{aligned} G_0 &= g_3 + (p_3 \cdot g_2) + (p_3 \cdot p_2 \cdot g_1) + (p_3 \cdot p_2 \cdot p_1 \cdot g_0) \\ G_1 &= g_7 + (p_7 \cdot g_6) + (p_7 \cdot p_6 \cdot g_5) + (p_7 \cdot p_6 \cdot p_5 \cdot g_4) \\ G_2 &= g_{11} + (p_{11} \cdot g_{10}) + (p_{11} \cdot p_{10} \cdot g_9) + (p_{11} \cdot p_{10} \cdot p_9 \cdot g_8) \\ G_3 &= g_{15} + (p_{15} \cdot g_{14}) + (p_{15} \cdot p_{14} \cdot g_{13}) + (p_{15} \cdot p_{14} \cdot p_{13} \cdot g_{12}) \end{aligned}$$

Figure A.6.2 updates our plumbing analogy to show P_0 and G_0 .

Then the equations at this higher level of abstraction for the carry in for each 4-bit group of the 16-bit adder (C_1, C_2, C_3, C_4 in Figure A.6.3) are very similar to the carry out equations for each bit of the 4-bit adder (c_1, c_2, c_3, c_4) on page A-40:

$$\begin{aligned} C_1 &= G_0 + (P_0 \cdot c_0) \\ C_2 &= G_1 + (P_1 \cdot G_0) + (P_1 \cdot P_0 \cdot c_0) \\ C_3 &= G_2 + (P_2 \cdot G_1) + (P_2 \cdot P_1 \cdot G_0) + (P_2 \cdot P_1 \cdot P_0 \cdot c_0) \\ C_4 &= G_3 + (P_3 \cdot G_2) + (P_3 \cdot P_2 \cdot G_1) + (P_3 \cdot P_2 \cdot P_1 \cdot G_0) \\ &\quad + (P_3 \cdot P_2 \cdot P_1 \cdot P_0 \cdot c_0) \end{aligned}$$

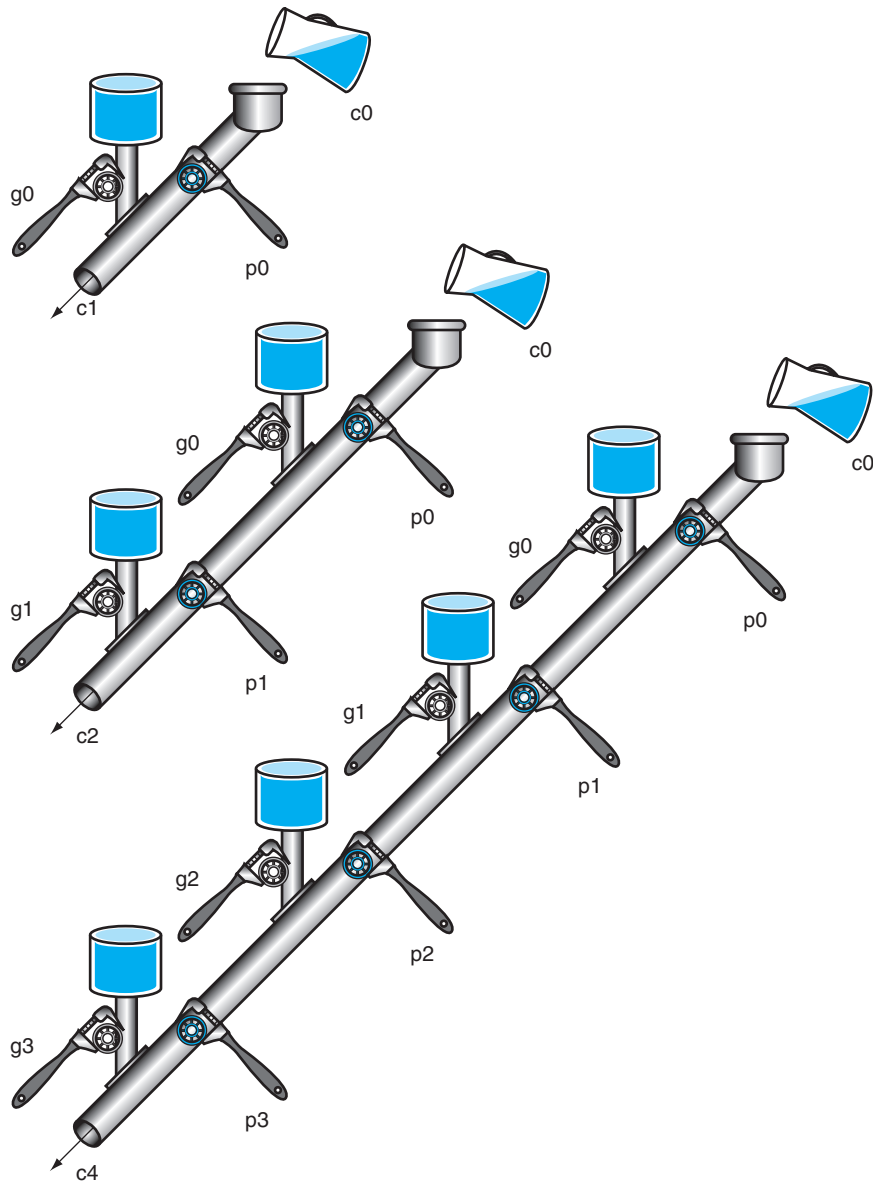


FIGURE A.6.1 A plumbing analogy for carry lookahead for 1 bit, 2 bits, and 4 bits using water pipes and valves. The wrenches are turned to open and close valves. Water is shown in color. The output of the pipe ($c_i + 1$) will be full if either the nearest generate value (g_i) is turned on or if the i propagate value (p_i) is on and there is water further upstream, either from an earlier generate or a propagate with water behind it. CarryIn (c_0) can result in a carry out without the help of any generates, but with the help of *all* propagates.